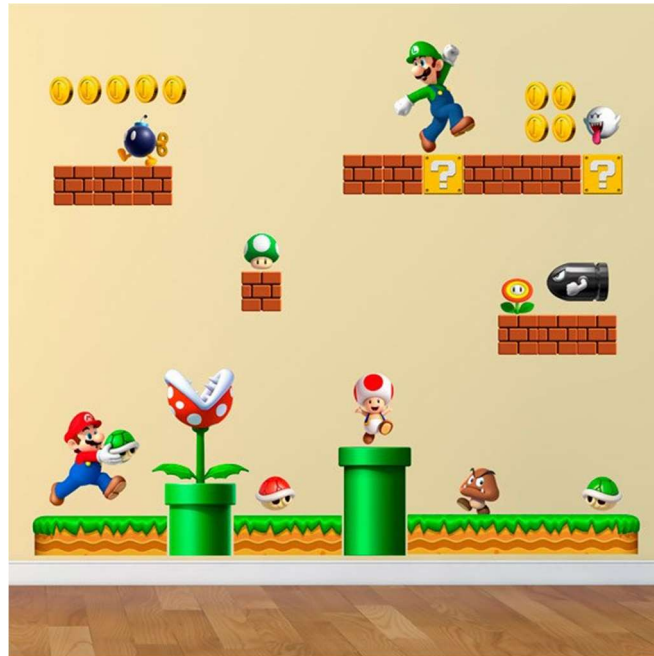


CONCEPTION



STRING AVENTURE

Type	Conception – S2
Nom du projet	STRING AVENTURE
Commentaire	S2, ENIB
Auteur	Turpin Antoine (a2turpin@enib.fr)
Version	6.0
Date	06/04/2023

Table des matières

1 Rappel du cahier des charges	4
1.1 Contraintes techniques	4
1.2 Fonctionnalités	4
1.3 P2 :Prototype P2.....	5
2 Principes des solutions techniques adoptées	5
2.1 Langage.....	5
2.2 Architecture du logiciel	5
2.3 Interface utilisateur	5
2.3.1 Boucle de simulation	5
2.3.2 Affichage	5
2.3.3 Gestion du clavier.....	5
2.3.4 Image ascii-art	5
3 Analyse de conception	6
3.1 Analyse noms/verbes :	6
3.2 Types de donnée	6
3.3 Dépendance entre modules	7
3.4 Analyse descendante :	7
3.4.1 Arbre principal :	7
3.4.2 Arbre affichage	7
3.4.3 Arbre interaction	7
4 Description des fonctions	8
4.1 Programme Principal : Main.py	8
4.2 Game.py	8
4.3 Monster.py	9
4.4 Player.py.....	10
4.5 Level.py	11

4.6 Animation.py	13
5 Calendrier et suivi de développement	13
5.1 P1 :	13
5.1.1 fonctions à développer	13
5.1.2 autre	14
5.2 P2 :	14
5.2.1 fonctions à développer	14

1 Rappel du cahier des charges

1.1 Contraintes techniques

- Le logiciel est associé à un cours, il doit donc fonctionner sur les machines de TP de l'ENIB pour que les élèves puissent le tester.
- Le langage utilisé en cours est Python. Le développement devra donc se faire en python.
- Les notions de programmation orientée objet n'ayant pas encore été abordées, le programme devra essentiellement s'appuyer sur le paradigme de la programmation procédurale.
- Le logiciel devra être réalisé en conformité avec les pratiques préconisées en cours de MDD : barrière d'abstraction, modularité, unicode, etc...
- L'interface sera réalisée en mode texte dans un terminal

1.2 Fonctionnalités

- F1 Jouer une partie
 - F1.1 Afficher le jeu
 - Écran
 - Héros
 - Ennemis
 - Plateformes et autres éléments
 - F1.2 Se déplacer / tirer
 - F1.3 Gérer collision héros et ennemis avec plateformes
 - F1.4 Gérer collision héros avec ennemis
 - F1.5 Finir un niveau
 - F1.5.1 Nouveau Niveau
 - F1.6 Finir partie plus de vie au héros
 - F1.6.1 Finir partie Défaite
- F2 Finir partie Victoire/Défaite
 - F2.1 Afficher écran Victoire ou Défaite
 - F2.2 Quitter

1.3 P2 : Prototype P2

Ce prototype réalise toutes les fonctionnalités.

Les fonctionnalités disponibles sont F1 et F2

Livré dans une archive au format .zip

Contient un manuel d'utilisation dans le fichier readme.txt

2 Principes des solutions techniques

2.1 Langage

Conformément aux contraintes énoncées dans le cahier des charges, le codage est réalisé avec langage python.

2.2 Architecture du logiciel

Nous mettons en œuvre le principe de la barrière d'abstraction. Chaque module correspond à un type de donnée et fournit toutes les opérations permettant de le manipuler de manière abstraite.

2.3 Interface utilisateur

L'interface utilisateur se fera via un terminal de type linux.

Nous reprenons la solution donnée en cours d'IPI en utilisant les modules :

`termios, sys, select.`

2.3.1 Boucle de simulation

Le programme mettra en œuvre une boucle de simulation qui gèrera l'affichage et les événements clavier.

2.3.2 Affichage

L'affichage se fait en communiquant directement avec le terminal en envoyant des chaînes de caractères sur la sortie standard de l'application.

2.3.3 Gestion du clavier

L'entrée standard est utilisée pour détecter les actions de l'utilisateur.

Le module `tty` permet de rediriger les événements clavier sur l'entrée standard.

Pour connaître les actions de l'utilisateur il suffit de lire l'entrée standard.

2.3.4 Image ascii-art

Pour dessiner certaines parties de l'interface nous utilisons des « images ascii ».

Dans l'idée de séparer le code et les données, les différentes images ASCII seront stockées dans des fichiers textes : `start.txt`, `END.txt`, `level.txt`

3 Analyse

3.1 Analyse noms/verbes :

- Verbes :

Jouer, afficher, déplacer, chuter, sauter, tirer, sauter, arriver, quitter

- Nom :

Terrain, héros, ennemis, plateforme, partie, résultats

3.2 Types de donnée

```
type : Player = struct
  x : float
  y : float
  speed : float
  shoot : list
  hp : int
  h : int
  last_move : str

  fstruct

type : Monster = struct
  m : list
  m : [ [type,x,y,hp,fire,cooldown,
        lim,xf,yf,direction,speed] ]

  fstruct

type : Level = struct
  m : Monster object
  levels : list
  indice : int
  grid : list

  fstruct

type : Game = struct
  lvl : Level object
  p : Player object

  fstruct

type : Animation = struct
  a :dict()
  a["x"] : int
  a["y"] : int
  a["frame"] : list()
  a["on"] : Bool
  a["timeLeft"] : float
  a["duration"] : float
  a["dt"] : float
```

3.3 Dépendance entre modules

Main dépend de Game et d'Animation

Game dépend de Level et de Player

Level dépend de Monster et Animation

3.4 Analyse descendante :

3.4.1 Arbre principal :

```
Main.init()  
|   +-- Animation.create() #*2, animation de début et de fin  
|   +-- Game.create()  
|       +--Level.create()  
|           +--Monster.create()  
|           +--Player.create()  
|  
+-- Main.run ()  
    +-- Animation.setOn()  
        +--Animation.show  
    +-- Main.interact()  
    +-- Game.live()  
    +-- Game.show()
```

3.4.2 Arbre affichage

```
Main.show()  
    +-- Game.show()  
    |   +-- Level.show()  
    |       +-- Monster.show()  
    |   +-- Player.show()
```

3.4.3 Arbre interaction

```
Main.interact()  
|  
+-- Main.isData()  
    +-- Game.quitGame()  
    +-- Game.left()  
    +-- Game.right()  
    +-- Game.jump()  
    +-- Game.down()  
    +-- Game.fire()
```

4 Description des fonctions

Note : Dans cette partie je n'ai pas décrit le fonctionnement de mes Accesseurs et de mes Mutateurs

4.1 Programme Principal : Main.py

- `Main.init()`
- `Main.run ()`
- `Main.interact()`
- `Main.quitGame()`

`Main.init ()`

Description : initialisation du jeu
Paramètres : aucun
Valeur de retour : g

`Main.run (g)`

Description : boucle de simulation
Paramètres : aucun
Valeur de retour : aucune

`Main.interact ()`

Description : gère les événements clavier
Paramètres : aucun
Valeur de retour : aucune

`Main.isData ()`

Description : vérifie la présence d'évènements clavier

`Main.finish ()`

Description : termine la partie, affiche l'animation de fin
Paramètres : aucun
Valeur de retour : aucune

4.2 Game.py

- `Game.create()`
- `Game.show(g)`
- `Game.live(g,dt)`
- `Game.collission(g,xy)`
- `Game.jump(g,dt)`
- `Game.down(g)`
- `Game.getHp(g)`
- `Game.left(g,dt)`
- `Game.right(g,dt )`
- `Game.fire(g)`

`Game.create ()`

Description : crée une nouvelle partie
level :Level.create()

player : Player.create()

Valeur de retour : g

Game.**show(g)**

Description : affiche le visuel de la partie en cours

Paramètres :

g : Game

Game.**live(g)**

Description : effectue tous les calculs de collisions pour les chutes, contact ect

Paramètres :

g : Game

Valeur de retour : rien

Game.**collision(g,x,y)**

Description : retourne l'élément présent aux coordonnées indiqués en paramètres

Paramètres :

g : Game x : int y : int

Valeur de retour : str

Game.**jump ()**

Description : permet au joueur de sauter en fonction des collisions

Valeur de retour : rien

Game.**down ()**

Description : permet au joueur de descendre d'une case

Valeur de retour : rien

Game.**getHp ()**

Description : retourne les hp restants au joueur

Valeur de retour : int

Game.**left ()**

Description : permet au joueur de se déplacer vers la gauche

Valeur de retour : rien

Game.**right ()**

Description : permet au joueur d'aller vers la droite

Valeur de retour : rien

Game.**fire ()**

Description : appel la fonction Player.fire()

Valeur de retour : rien

4.3 Monster.py

- Monster.**create()**
- Monster.**show(m,i)**
- Monster.**newMonster(m,i)**
- Monster.**move(m,i,x,y)**
- Monster.**chute(m,i,dt)**

- `Monster.initShoot(m,i)`
- `Monster.posShoot(m)`
- `Monster.Shoot(m,i)`
- `Monster.testFull(m)`

`Monster.create ()`

Description : Crée une nouvelle liste d'ennemis

Paramètres : rien

Valeur de retour : nouveaux monstres

`Monster.show(m,i)`

Description : affiche les monstres

Paramètres :

m : Monstre

Valeur de retour : rien

`Monster.initShoot(m,i)`

Description : initialise un projectile

Valeur de retour : rien

`Monster.Shoot(m,i,dt)`

Description : modifie la position du projectile crée

Valeur de retour : rien

`Monster.testFull()`

Description : teste si la liste de monstre est pleine et les fonctions associées

Paramètres :

m : monstres

4.4 Player.py

- `Player.create()`
- `Player.move(p,x,y)`
- `Player.jump(p,dt)`
- `Player.chute(p,dt)`
- `Player.showShoot(p,i)`
- `Player.newShoot(p)`
- `Player.fire(p)`
- `Player.getposFire(p)`
- `Player.live(p)`
- `Player.testFull(p)`
- `Player.show(p)`

`Player.create () -> Player`

Description : Crée un nouveau joueur

Paramètres : rien

Valeur de retour : nouveau joueur

`Player.jump (p,dt)`

Description : permet au joueur de sauter (saut limité)

Paramètres : p : Player

Valeur de retour : rien

`Player.fire(p,i)`

Description : permet au joueur de tirer des projectiles dans la direction choisie (droite ou gauche)
Paramètres : p : Player, i : indice du projectile dans la liste
Valeur de retour : rien

Player.showShoot(p,i)

Description : Stop l'affichage du projectile ciblé
Paramètres : p : Player, i : indice du projectile
Valeur de retour : rien

Player.newShoot(p)

Description : forme un nouveau projectile sur la position du joueur
Paramètres : p : Player
Valeur de retour : i : indice du projectile crée

Player.move(p,x,y)

Description : change la position du joueur donné (une fois la collision vérifiée dans Game)
Paramètres : p : Player, x et y : int nouvelle position
Valeur de retour : rien

Player.live(p,dt)

Description : calcul les nouvelles positions des projectiles du joueur
Paramètres : p : Player, dt : TimeStep du jeu
Valeur de retour : rien

Player.show(p)

Description : affiche le joueur et ses projectiles
Paramètres :
p : Player
Valeur de retour : rien

Player.testFull(p)

Description : teste si les fonctions associées à la structure de Player fonctionnent
Paramètres :
p : Player

4.5 Level.py

- Level.create(levels,i,dt)
- Level.parse(lvl,filename)
- Level.show(lvl)
- Level.Hp_1(i)
- Level.pos_monster(lvl)
- Level.live(lvl,dt)
- Level.posShoot()
- Level.delShoot(lvl,i)
- Level.Newlvl(lvl)
- Level.testFull()

Level.create (levels,i,dt)

Description : Crée un objet Level() et initialise les monstres et une Animation

Paramètres : levels : liste de nom des fichiers de niveaux, i : indice du niveau actuel

Valeur de retour : nouveaux niveaux

Level.parse (lvl,filename)

Description : Récupère les infos d'un fichier.txt et le découpe pour obtenir une liste de liste d'éléments correspondant à notre grille avec ses coordonnées

Paramètres : lvl : Level, filename : nom du fichier à découper

Valeur de retour : retourne lvl.grid : liste

Level.show (lvl)

Description : affiche le terrain

Paramètres :

lvl : Level

Valeur de retour : rien

Level.Hp_1(i)

Description : retire 1 hp à un monstre (d'après l'indice i)

Paramètres : i : indice monstre

Valeur de retour : rien

Level.pos_monster (lvl)

Description : retourne la position de tous les monstres

Paramètres : lvl : Level

Valeur de retour : l : liste

Level.live (lvl,dt)

Description : calcul les positions des monstres et des projectiles associés

Paramètres : lvl : Level, dt : TimeStep du jeu

Valeur de retour : rien

Level.posShoot ()

Description : retourne toutes les positions des projectiles

Paramètres : rien

Valeur de retour : Monster.posShoot(m) : liste

Level.delShoot(lvl,i)

Description : supprime un projectile indiqué

Paramètres : lvl : Level, i : indice projectile

Valeur de retour : rien

Level.Newlvl(lvl)

Description : initialise le niveau suivant et lance une animation en cas de fin de jeu

Paramètres : lvl : Level

Valeur de retour : lvl : Level

Level.testFull(lvl) :

Description : teste si les fonctions associées à la structure de Level fonctionnent

Paramètres : rien

Valeur de retour : rien

4.6 Animation.py

- **Animation.create(filename,dt)**
- **Animation.parse(a,filename)**
- **Animation.show(a)**
- **Animation.nextFrameIndex(a)**

Animation.create(filename,dt) :

Description : initialisation d'une animation

Paramètres : filename : nom du fichier, dt : TimeStep du jeu

Valeur de retour : a : Animation

Animation.parse(a,filename) :

Description : découpe le fichier filename

Paramètres : a : Animation, filename : nom du fichier

Valeur de retour : rien

Animation.show(a) :

Description : affichage de l'animation

Paramètres : a : Animation

Valeur de retour : rien

Animation.nextFrameIndex(a) :

Description : calcul l'indice (pour l'affichage) en fonction du rafraichissement

Paramètres : rien

Valeur de retour : rien

5 Calendrier et suivi de développement

5.1 P1 :

5.1.1 Fonctions à développer

fonctions	commentaires
Main.init()	
Main.run()	
Main.show()	
Main.interact()	
Main.finish()	
Game.create()	
Game.show(g)	
Game.live(g)	
Game.collisioin (g,x,y)	
Game.finish(g)	

Monster.create()
Monster.testFull(m)
Monster.show(m)
Monster.Shoot(m)
Monster.initShoot(m,i)
Monster.move(m,i,x,y)
Player.create()
Player.move(p,x,y)
Player.testFull(p)
Player.show(p)
Level.create()
Level.parse(lvl)
Level.testFull(lvl)
Level.show(lvl)
Player.jump(p)

5.1.2 autre

level1.txt , level2.txt , level3.txt

5.2 P2 :

5.2.1 Fonctions à développer

Fonctions	commentaires
Player.fire(p)	
Animation.create()	
Animation.parse()	
Animation.show(	
Animation.nextFrameIndex()	
Player.fire()	
Player.showShoot()	
Player.nexShoot()	

Suite à la première partie du développement, en ayant un jeu fonctionnel bien qu'imparfait et pas fini, pendant la seconde période j'ai évidemment ajouté du contenu dans mon jeu mais j'ai surtout :

- Réparer les bugs
- Changer la façon dont j'ai programmé par endroit (refaire le Level.py, retirer le Fire.py qui était inutile ect)
- **Optimiser mon programme** afin de diminuer les calculs et les lignes inutiles